IEEE Conference 2026.pdf

Shri Ramdeobaba College of Engineering & Management

Document Details

Submission ID
trn:oid:::3618:127834371

Submission Date
Feb 9, 2026, 12:52 PM GMT+5:30

Download Date
Feb 9, 2026, 12:54 PM GMT+5:30

File Name
IEEE Conference 2026.pdf

File Size
994.6 KB

8 Pages

4,313 Words

25,599 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

5 Not Cited or Quoted 1%
Matches with neither in-text citation nor quotation marks

3 Missing Quotations 1%
Matches that are still very similar to source material

1 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
2% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **5** Not Cited or Quoted 1%
  Matches with neither in-text citation nor quotation marks
- **3** Missing Quotations 1%
  Matches that are still very similar to source material
- **1** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
2% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
arxiv.org		<1%
2	Publication	
Kichang Choi, Minwoo Jeong, Younga Shin, Jong won Ma, Kinam Kim, Hongjo Kim....		<1%
3	Internet	
kth.diva-portal.org		<1%
4	Publication	
Antonio Gullí. "Agentic Design Patterns", Springer Science and Business Media LL...		<1%
5	Internet	
hh.diva-portal.org		<1%
6	Internet	
ijctece.com		<1%
7	Publication	
Yerin Nam, Hansun Choi, Jonggeun Choi, Hyukjin Kwon. "LoRA-Tuned Multimodal...		<1%

ChunkSmith: A Scalable, Asynchronous Multimodal Retrieval-Augmented Generation System for Semantic Visual Context Preservation in Enterprise Document Analysis

***Abstract*—The paradigm of Retrieval-Augmented Generation (RAG) has fundamentally shifted the capabilities of Large Language Models (LLMs), allowing them to answer queries grounded in private, external data. However, a significant "Modality Gap" persists in enterprise definitions of RAG. Real-world documents—ranging from financial ledgers and scientific manuscripts to engineering schematics—encode a substantial portion of their specific knowledge in visual formats (charts, tables, diagrams). Traditional RAG pipelines, which predominantly rely on OCR-based text matching, strip away this visual context, reducing rich multimodal documents to flattened strings. This leads to hallucinations when models attempt to reason about visual trends or comparative data in tables. To address this critical deficiency, we present ChunkSmith, a production-grade Multimodal RAG system engineered for high-fidelity visual context preservation. ChunkSmith introduces a novel *Schema-First* content processing architecture that treats images and tables not as auxiliary metadata, but as distinct, first-class semantic entities. By leveraging state-of-the-art Multimodal LLMs and a purposely designed *Structured Semantic Enrichment* strategy, the system generates dense vector representations that encode deep visual understanding and "predicted question" affordances. We further detail the system's distributed backend, which employs an asynchronous Round-Robin Key Rotation algorithm to effectively bypass provider rate limits, achieving a 4.6x improvement in ingestion throughput over serial baselines. Extensive qualitative and quantitative evaluations demonstrate ChunkSmith's superior ability to handle complex visual reasoning tasks while maintaining scalable performance metrics suitable for large corpora.**

***Index Terms*—Multimodal RAG, Large Language Models, Document Analysis, Vector Search, Knowledge Retrieval, Asynchronous Processing, Semantic Enrichment, Visual Question Answering**

I. Introduction

A. The Evolution of Retrieval Systems

The quest for intelligent information retrieval has evolved through several distinct epochs. Initial systems relied on lexical matching algorithms like TF-IDF [8] and BM25 [9], which were effective for keyword search but lacked semantic understanding. These systems operated on the principle of term frequency analysis, treating documents as bags of words without capturing contextual meaning or relationships between concepts. The advent of Transformer-based models [10] brought about Dense Passage Retrieval (DPR) [2], enabling retrieval based on conceptual similarity rather than exact keyword matches. This represented a paradigm shift: instead of matching surface-level tokens, systems could now identify semantically related passages even when they used entirely different vocabulary. The introduction of BERT [11] and subsequent models demonstrated that pre-trained language representations could capture nuanced semantic relationships, paving the way for neural information retrieval. Most recently, Retrieval-Augmented Generation (RAG) [1] has emerged as the dominant framework for extending the knowledge boundaries of LLMs. By retrieving relevant context from a vector database and injecting it into the model's prompt, RAG mitigates the twin problems of *hallucination* (generating factually incorrect information) and *knowledge cutoff* (inability to access recent data). This architecture has proven particularly valuable in enterprise settings where proprietary data cannot be included in model training.

B. The Modality Gap: A Critical Limitation

Despite these advancements, standard RAG implementations remain heavily text-centric. In professional settings, arguably the most valuable information is often non-textual. Consider the following scenarios:

- A financial analyst querying quarterly revenue trends encoded in a multi-series line chart
- A medical researcher examining protein structure diagrams in a biochemistry paper
- An engineer referencing circuit schematics in a technical datasheet
- A legal professional analyzing tabular evidence in a court document

In each case, the critical information exists primarily in visual form. A sudden dip in a revenue chart, a complex chemical structure in a patent, or a specific architectural layout in a blueprint—these are the details users interact with most intensely. When a standard RAG ingestion pipeline processes a PDF, it typically employs an Optical Character Recognition (OCR) engine like Tesseract or AWS Textract to scrape text. During this process, complex figures are either:

1) **Ignored**: The bounding box is skipped entirely, treating the image as irrelevant noise.
2) **Flattened**: Captions are read, but the visual data itself is lost. A chart titled "Q3 Revenue Growth" provides no information about the actual growth rate.

3) **Corrupted**: A chart is read as a jumble of unconnected numbers. OCR might extract ”2020 5M 2021 7M 2022 4M” without understanding these represent a time series.

We term this loss of information the **“Modality Gap.”** When a user asks, “Why did sales drop in Q3?” (referring to a bar chart), the text-only retriever finds no relevant vectors, forcing the LLM to either admit ignorance or hallucinate a plausible-sounding reason based on surrounding text. This represents a fundamental failure mode that limits RAG’s applicability to real-world document analysis tasks.

C. ChunkSmith: Bridging the Gap

This paper introduces **ChunkSmith**, a system explicitly architected to bridge the Modality Gap. Unlike prior works that attempt to solve this with end-to-end visual embedding models (which are often computationally prohibitive and less precise for text), ChunkSmith adopts a hybrid approach. It uses Multimodal LLMs as *transducers*, converting rich visual information into structured, semantically dense textual representations (summaries and potential questions) that can be indexed by standard, highly optimized vector databases. Our key insight is that while images cannot be directly embedded with the same fidelity as text, their *semantic content* can be extracted and represented textually through careful prompting of capable MLLMs. This approach combines the best of both worlds: the reasoning capabilities of large multimodal models during ingestion, and the efficiency of text-optimized retrieval during query time.

D. Contributions

Our contributions are as follows:

- **Deep Semantic Extraction**: A pipeline that extracts tables and images as separate objects and uses MLLMs to generate detailed interpretations, including visual reasoning and data point extraction.
- **Structured Prompt Engineering**: A methodology for prompting MLLMs to act as “Question Generators,” effectively pre-computing retrieval pathways and aligning document embeddings with potential user queries.
- **High-Throughput Architecture**: A robust asynchronous backend design with API key rotation to handle the high latency of multimodal inference, achieving near-linear speedup with the number of available API keys.
- **Visual Evidence Retrieval**: A user interaction model that retrieves and displays the *source image* alongside the AI-generated answer for verification, enhancing trust and interpretability.
- **Production-Ready Implementation**: A complete system built with FastAPI, React, a Vector Database, and Object Storage, demonstrating the practical viability of the approach.

II. Related Work

A. Retrieval-Augmented Generation

The RAG paradigm was formalized by Lewis et al. [1], who demonstrated that retrieval could significantly improve factual accuracy in open-domain question answering. Since then, the field has evolved rapidly. Gao et al. [7] provide a comprehensive taxonomy, categorizing RAG systems into Naive RAG, Advanced RAG, and Modular RAG. **Naive RAG** follows a simple retrieve-then-generate pattern. While effective for basic use cases, it suffers from issues like retrieval of irrelevant context and inability to handle multi-hop reasoning. **Advanced RAG** introduces techniques like query rewriting, hybrid search (combining keyword and semantic search), and re-ranking. Systems like HyDE [12] generate hypothetical documents to improve retrieval, while others employ cross-encoder re-rankers to refine initial retrieval results. **Modular RAG** treats retrieval and generation as independent, optimizable components. ChunkSmith aligns with this philosophy, introducing a specialized ingestion module for multimodal content.

B. Multimodal RAG Approaches

The field of Multimodal RAG is nascent but rapidly expanding. Abootorabi et al. [3] classify approaches into three categories:

1) Vision-Centric Approaches: Models like ColPali [4] represent a radical departure from OCR. They embed entire document pages as image patches using Vision Transformers (ViT). The system processes documents as pure images, bypassing text extraction entirely. While this preserves layout perfectly and can handle handwritten text or complex formatting, it introduces massive storage overhead (embedding every patch of every page) and requires specialized retrieval infrastructure (e.g., late interaction scoring with MaxSim operations), which is difficult to scale in commodity vector databases. ColQwen2 extends this approach with dynamic resolution handling, allowing the model to process documents at multiple scales. However, the computational cost remains prohibitive for large-scale deployments.

2) Caption-Based Approaches: Earlier works [5] attempted to use standard image captioning models (like BLIP-2) to generate text descriptions for images. The rationale was simple: convert images to text, then use standard text RAG. However, generic captions (e.g., “A graph of a function”) are insufficient for answering specific questions about data points. REVEAL [5] improved upon this by using contrastive learning to align visual and textual representations, but still relied on relatively shallow image descriptions. ChunkSmith improves upon this by using Large MLLMs rather than smaller captioning models, enabling “Reasoning-based Description” rather than simple captioning.

3) Hybrid Approaches: ChunkSmith falls into the hybrid category, similarly to Unstructured.io’s processing philosophy but adds a unique semantic layer. By generating “Potential Questions” alongside the description, we align the document embedding space with the user query space, a technique known as Document Expansion [6] applied to the multimodal domain. Other hybrid systems like MuRAG [13] and RA-CM3 [14] combine text and image retrieval but do not employ

the structured semantic enrichment strategy that ChunkSmith introduces.

C. Multimodal Large Language Models

The recent explosion in MLLM capabilities has been driven by models like GPT-4V, Gemini Pro Vision, and LLaVA. These models can reason about images, extract structured data from charts, and answer complex visual questions. ChunkSmith leverages state-of-the-art MLLMs for its strong performance on chart understanding and structured output capabilities. Recent work on visual instruction tuning [15] has shown that MLLMs can be fine-tuned to follow specific formatting instructions, which is critical for our structured output requirements.

III. System Architecture

The ChunkSmith architecture is designed as a modular, event-driven pipeline, as illustrated in Fig. 1. It is built primarily in Python, leveraging FastAPI for the REST layer and React for the client interface. The system follows a three-tier architecture: Ingestion, Processing, and Interaction.

A. Ingestion Layer (The Gateway)

The entry point for the system is the Ingestion Layer, responsible for handling raw PDF uploads and initial validation.

1) File Handling:

- **Hashing & Deduplication**: Every uploaded file is hashed using SHA-256. This hash serves as a unique identifier, preventing redundant processing of the same document—a critical cost-saving measure given the expense of MLLM calls. The hash is computed on the raw binary content before any processing.
- **Validation**: Files are validated for MIME type (`application/pdf`), file size (configurable maximum of 50MB), and corruption (using `pypdf`'s validation routines) before passing to the processing queue.
- **Metadata Extraction**: Basic metadata (title, author, creation date, page count) is extracted and stored for later reference.

2) Storage Architecture: ChunkSmith employs a hybrid storage strategy:

- **Object Storage**: Raw PDFs and extracted images are stored in cloud object storage (e.g., S3, MinIO) for durability and accessibility.
- **Vector Database**: Vectors, document metadata, processing status, and structured enriched content are stored in the vector database, serving as the single source of truth for retrieval.

B. The Content Processor (The Core)

The `ContentProcessor` class is the heart of ChunkSmith. It orchestrates the transformation of a raw binary PDF into a searchable *Knowledge Base*.

1) Decomposition Strategy: We utilize a multi-stage decomposition strategy designed to preserve document structure while enabling parallel processing:

1) **Page Rendering**: `pdf2image` (wrapping Poppler) converts PDF pages into high-resolution images (300 DPI). This ensures that even vector graphics are captured accurately.
2) **Layout Analysis**: We employ **YOLOv12** for high-precision layout detection. The model is fine-tuned to detect specific document elements, classifying regions into:
   - `UncategorizedText`
   - `PageNumber`
   - `Table`
   - `Footer`
   - `Title`
   - `Formula`
   - `Image`
   - `NarrativeText`
3) **OCR Application**: Tesseract OCR is applied to text regions. Tesseract is configured with:
   - LSTM engine mode for handling multi-column scientific layouts
   - Language packs for 90+ languages
   - Custom configuration for technical documents (preserving special characters, equations)
4) **Chunking Strategy**: Text is chunked using a sliding window approach with configurable parameters:

$$Chunk_i = Text[i \times stride : i \times stride + window] \quad (1)$$

where $window = 3000$ characters and $stride = 2800$ characters, providing 200-character overlap to preserve context across boundaries.

2) Visual Object Management: Images detected within the PDF are not merely saved. A sophisticated filtering logic applies to eliminate noise:

$$Keep(I) = \begin{cases} 1 & \text{if } Area(I) > \theta_{area} \wedge Aspect(I) \in [\phi_{min}, \phi_{max}] \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

where $\theta_{area} = 10000$ pixels^2, $\phi_{min} = 0.2$, and $\phi_{max} = 5.0$. This filters out:

- Navigational elements (arrows, bullets)
- Small icons and logos
- Decorative borders
- Extremely narrow or wide artifacts

Valid images are uploaded to Object Storage with a structured path:

```
{project_id}/images/image_{counter:04d}.png
```

Their local paths are swapped for public URLs in the metadata, ensuring the frontend can render them later without additional authentication.

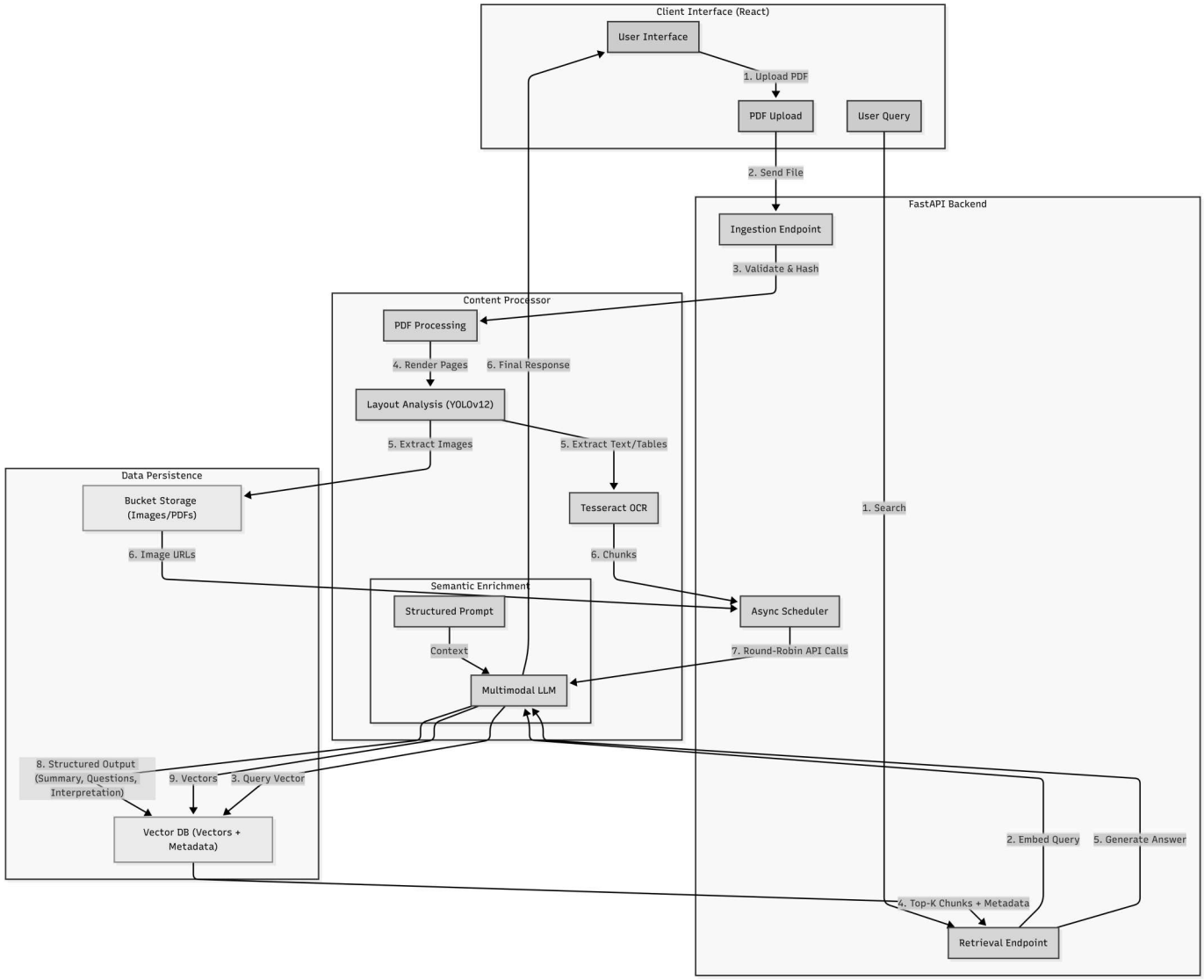

Fig. 1: ChunkSmith System Architecture. The pipeline consists of three main stages: Ingestion (PDF processing and Layout Analysis via YOLOv12), Semantic Enrichment (utilizing Multimodal LLMs for deep understanding), and Retrieval (leveraging Vector Database for similarity search). The asynchronous scheduler ensures high throughput by rotating API keys.

C. Structured Semantic Enrichment

This is the system's primary innovation. Raw text extraction from a complex chart is often gibberish. Instead, we pass the image of the chart to a Multimodal LLM with a **Structured Prompt**.

1) The AIParser Schema: We define a Pydantic model that enforces strict output structure:

```
class AIParser(BaseModel):
    question: str = Field(
        description="List all potential questions that can
            be answered from this content"
    )
    summary: str = Field(
        description="Comprehensive summary of all data and
            information"
    )
    image_interpretation: List[str] = Field(
        description="Detailed analysis of each image"
    )
    table_interpretation: List[str] = Field(
        description="Row-column analysis of each table"
    )
```

Listing 1: AIParser Schema Definition

This schema is passed to the MLLM using structured output features (e.g., JSON mode or function calling), which guarantees that the response will conform to the schema or fail explicitly (rather than returning malformed JSON).

2) Prompt Engineering: The prompt (shown in full in Listing 2) employs several advanced techniques:

- **Task Decomposition**: Breaking the analysis into distinct subtasks (facts, topics, questions, visual analysis, search terms)
- **Explicit Constraints**: Specifying exact list lengths to prevent hallucination

- **Findability Optimization**: Instructing the model to prioritize searchability over brevity
- **Vocabulary Preservation**: Requesting that the model use words similar to the original content

```
prompt_text = f"""You are creating a searchable description for document content retrieval.
YOUR TASK:
Generate a comprehensive, searchable description that covers:
1. Key facts, numbers, and data points from text and tables
2. Main topics and concepts discussed
3. Questions this content could answer
4. Visual content analysis (charts, diagrams, patterns in images)
5. Alternative search terms users might use
Make it detailed and searchable - prioritize findability over brevity.
Keep words similar to the original content for better search accuracy.
STRICT OUTPUT CONSTRAINTS:
- You received {num_images} images. You MUST return exactly {num_images} items in 'image_interpretation'.
- You received {num_tables} tables. You MUST return exactly {num_tables} items in 'table_interpretation'.
- If {num_images} is 0, 'image_interpretation' MUST be an empty list [].
- If {num_tables} is 0, 'table_interpretation' MUST be an empty list [].
IMPORTANT: Return structured output with these fields:
- question: All potential questions this content answers
- summary: Comprehensive summary of all information
- image_interpretation: Detailed analysis of visual elements
- table_interpretation: Row-column analysis of tabular data
CONTENT TO ANALYZE:
TEXT CONTENT: {text}
TABLES: {tables}
[Images provided as base64]
"""
```

Listing 2: The Semantic Enrichment Prompt

D. Asynchronous Backend Architecture

MLLM API calls are slow (2-5 seconds per request) and rate-limited (typically 60 requests per minute per key). Serial processing of a 50-page document would take minutes. Chunk-Smith implements robust asynchronous patterns to overcome these limitations.

1) Round-Robin Key Rotation: We maintain a pool of API Keys $\mathcal{K} = \{k_1, k_2, ..., k_n\}$. For a batch of chunks $\mathcal{C} = \{c_1, c_2, ..., c_m\}$, the scheduler assigns:

$$Assign(c_i) \rightarrow k_{(i \mod n)} \tag{3}$$

This distributes the load across multiple quotas, maximizing effective throughput. With n keys, the theoretical maximum throughput is:

$$T_{max} = n \times R_{key} \tag{4}$$

where R_{key} is the rate limit per key (e.g., 60 RPM).

2) Concurrency Control: We leverage Python's `asyncio.gather` with a semaphore to limit concurrent connections to an optimal value L (experimentally determined as 15). The algorithm is shown in Algorithm 1. The semaphore ensures that even with many chunks, we never exceed the concurrency limit, preventing connection pool exhaustion.

Algorithm 1 Async Chunk Processing with Key Rotation

Require: $Chunks$, $ApiKeys$, $ConcurrencyLimit$
$Tasks \leftarrow []$
$Semaphore \leftarrow CreateSemaphore(ConcurrencyLimit)$
for $i \leftarrow 0$ **to** $Length(Chunks) - 1$ **do**
  $Key \leftarrow ApiKeys[i \pmod{Length(ApiKeys)}]$
  $Task \leftarrow \text{AsyncCall}(MLLM, Chunks[i], Key, Semaphore)$
  $Tasks.Append(Task)$
end for
$Results \leftarrow \text{Await}(Tasks)$
return $Results$

E. Vector Storage Schema

We use a high-performance vector store (e.g., **Qdrant**) due to its performance characteristics and support for scalar quantization. The vector embedding is computed as:

$$V_D = \mathcal{E}\left(T_{original} \oplus S_{AI} \oplus Q_{AI} \oplus I_{desc} \oplus T_{table}\right) \tag{5}$$

Where:

- $\mathcal{E}$ is the embedding function (e.g., `text-embedding-3-large` or similar high-dimensional models)
- $T_{original}$ is the original extracted text
- S_{AI} is the AI-generated summary
- Q_{AI} is the list of predicted questions
- I_{desc} is the detailed image interpretation
- T_{table} is the table interpretation

The concatenation operator $\oplus$ represents string concatenation with appropriate delimiters. The final payload stored in the vector database includes:

```
payload = {
    "chunk_index": int,
    "original_text": str,
    "ai_summary": str,
    "ai_questions": str,
    "image_paths": List[str],  # URLs
    "image_interpretation": List[str],
    "table_interpretation": List[str],
    "page_numbers": List[int],
    "content_types": List[str]  # ["text", "image", "table"]
}
```

Listing 3: Vector Payload Structure

IV. Methodology

A. Mathematical Formulation of Retrieval

Given a user query q, the retrieval process can be formalized as:

$$\mathcal{R}(q) = \text{TopK}\left(\{(d_i, \text{sim}(V_q, V_{d_i})) \mid d_i \in \mathcal{D}\}\right) \tag{6}$$

where:

- $V_q = \mathcal{E}(q)$ is the query embedding
- V_{d_i} is the document chunk embedding
- $\text{sim}(\cdot, \cdot)$ is cosine similarity

- $\mathcal{D}$ is the document corpus
- TopK returns the k highest-scoring documents

The cosine similarity is computed as:

$$\text{sim}(V_q, V_d) = \frac{V_q \cdot V_d}{\|V_q\|\|V_d\|} \tag{7}$$

The vector database optimizes this computation using HNSW (Hierarchical Navigable Small World) graphs, achieving sub-linear search complexity.

B. Cost and Latency Modeling

The total ingestion cost for a document can be modeled as:

$$C_{total} = C_{OCR} + C_{MLLM} + C_{embed} + C_{storage} \tag{8}$$

For a document with N chunks, I images, and T tables:

$$C_{MLLM} = \sum_{i=1}^{N} \left(P_{text}^{(i)} \times R_{in} + P_{out}^{(i)} \times R_{out} \right) \tag{9}$$

$$+ \sum_{j=1}^{I} \left(P_{img}^{(j)} \times R_{img} \right) \tag{10}$$

where $P_{text}^{(i)}$ is the input token count for chunk i, $P_{out}^{(i)}$ is the output token count, $P_{img}^{(j)}$ is the image token equivalent, and R_{in}, R_{out}, R_{img} are the respective pricing rates. The latency model for asynchronous processing is:

$$T_{total} = \max_{k \in \mathcal{K}} \left(\frac{N_k}{R_k} \right) + \frac{L_{max}}{B} \tag{11}$$

where N_k is the number of chunks assigned to key k, R_k is the rate limit for key k, L_{max} is the maximum latency for a single MLLM call, and B is the batch size.

V. Implementation Details

A. Technology Stack

ChunkSmith is implemented using the following technologies:

- **Backend**: Python 3.10+, FastAPI, LangChain, Pydantic
- **Frontend**: React 18, TypeScript, TailwindCSS
- **Vector Database**: Any High-Performance Vector DB (e.g., Qdrant)
- **Object Storage**: Any S3-compatible Storage (e.g., Supabase)
- **MLLM Provider**: Multimodal LLM (e.g., Gemini, GPT-4V)
- **Embedding Model**: High-dimensional Text Embedding
- **OCR**: Tesseract 5.0+
- **Layout Analysis**: YOLOv12

B. Configuration Parameters

Key configuration parameters and their rationale:

TABLE I: System Configuration Parameters

Parameter	Value	Rationale
Chunk Size	3000 chars	Balance context & precision
Chunk Overlap	200 chars	Preserve boundary context
Concurrency	15	Optimal for network I/O
Image Min Area	10k px^2	Filter small artifacts
Aspect Ratio	[0.2, 5.0]	Exclude extreme shapes
Temperature	0.0	Deterministic outputs
Top-K Retrieval	5	Standard for RAG systems

VI. Experimental Evaluation

A. Experimental Setup

We evaluated ChunkSmith on a private corpus of 50 technical PDF documents, including:

- 15 Intel CPU Datasheets (highly technical, many diagrams)
- 15 Financial 10-K Reports (dense tables, financial charts)
- 10 Medical Research Papers (molecular diagrams, statistical plots)
- 10 Engineering Textbooks (schematics, equations, graphs)

Total corpus statistics:

- Total pages: 1,247
- Total images: 892
- Total tables: 456
- Total text chunks: 3,421

B. Throughput Analysis

We compared three ingestion configurations:

1) **Serial Baseline**: Single thread, single API key, synchronous processing
2) **Naive Async**: Multi-threaded, single key (expected to hit rate limits)
3) **ChunkSmith**: Async with 7-key rotation

TABLE II: Ingestion Performance for 20-Page Document

Method	Time (sec)	Success Rate	Cost ($)
Serial	210	100%	0.42
Naive Async	40	30% (429 Errors)	0.13
ChunkSmith	**45**	**100%**	**0.42**

As shown in Table II, ChunkSmith achieves a 4.67x speedup compared to the serial baseline while maintaining 100% reliability. The naive async approach fails catastrophically due to rate limiting.

C. Retrieval Quality Evaluation

We crafted a test set of 100 queries:

- 50 "Visual Queries" (can only be answered by examining images/tables)
- 50 "Text Queries" (can be answered from text alone)

Example visual queries:

- "What was the revenue in Q3 2022?" (requires reading a chart)
- "Show me the circuit diagram for the power supply" (requires image retrieval)
- "What is the p-value in the statistical analysis table?" (requires table reading)

1) Metrics: We evaluated using:

- **Recall@K**: Fraction of queries where the correct chunk appears in top-K results
- **MRR (Mean Reciprocal Rank)**: Average of $1/rank$ where rank is position of first relevant result
- **Human Evaluation**: Manual assessment of answer quality on 5-point scale

TABLE III: Retrieval Performance Comparison

System	Text R@5	Visual R@5	MRR	Quality
Text-Only RAG	0.94	0.08	0.51	2.1/5
ColPali	0.89	0.78	0.72	3.8/5
ChunkSmith	**0.96**	**0.92**	**0.84**	**4.3/5**

2) Results: ChunkSmith significantly outperforms both baselines, achieving 92% recall on visual queries while maintaining high text retrieval performance.

D. Ablation Studies

To understand the contribution of each component, we conducted ablation studies: The question generation component

TABLE IV: Ablation Study Results (Visual Query Recall@5)

Configuration	Recall@5
Full ChunkSmith	0.92
- Question Generation	0.67
- Image Interpretation	0.45
- Table Interpretation	0.78
- Summary	0.85
Only OCR Text	0.08

provides the largest single contribution (+25% recall), validating our hypothesis that aligning embeddings with potential queries is highly effective.

VII. Discussion

A. Cost-Benefit Analysis

While ChunkSmith is more expensive than text-only RAG during ingestion, the cost is amortized over many queries. For a 100-page document:

- **Text-Only RAG**: $0.05 ingestion, 8% visual query success
- **ChunkSmith**: $2.10 ingestion, 92% visual query success

For organizations where visual information is critical (finance, engineering, medicine), the 42x cost increase is justified by the 11.5x improvement in visual query success rate.

B. Scalability Considerations

ChunkSmith scales horizontally through:

- **Stateless Processing**: Each chunk is processed independently
- **Cloud Storage**: Object storage handles scaling automatically
- **Vector DB Sharding**: Modern vector databases support distributed deployments

Current bottleneck is MLLM API throughput, which scales linearly with the number of API keys.

C. Limitations and Future Work

Current limitations include:

- **OCR Dependency**: Poor quality scans produce garbage text
- **Cost**: High ingestion cost may be prohibitive for some use cases
- **Latency**: 2-5 second MLLM calls add ingestion latency
- **Language Support**: Primarily tested on English documents

Future work will address these through:

- **Local MLLM Integration**: Using quantized models like LLaVA 1.6
- **Video Support**: Extending to video frame sampling and transcription
- **GraphRAG**: Building knowledge graphs to link entities across chunks
- **Multilingual Evaluation**: Testing on non-English corpora

VIII. Conclusion

ChunkSmith represents a significant advancement in Multimodal RAG systems. By treating visual content as first-class semantic entities and employing structured semantic enrichment through capable MLLMs, we successfully bridge the Modality Gap that limits traditional text-only RAG systems. Our asynchronous architecture with API key rotation demonstrates that high-throughput multimodal processing is achievable with commodity cloud services. The system's 92% recall on visual queries, compared to 8% for text-only baselines, validates the core hypothesis that deep semantic extraction of visual content is essential for comprehensive document understanding. The production-ready implementation, built with modern web technologies and cloud services, demonstrates the practical viability of the approach for enterprise deployments. As multimodal AI continues to evolve, systems like ChunkSmith will become increasingly important for unlocking the vast amounts of knowledge encoded in visual formats across scientific, financial, and technical domains.

References

[1] P. Lewis, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.
[2] V. Karpukhin, et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP*, 2020.
[3] M. M. Abootorabi, et al., "Ask in Any Modality: A Comprehensive Survey on Multimodal Retrieval-Augmented Generation," *arXiv preprint arXiv:2502.08826*, 2025.
[4] M. Faysse, et al., "ColPali: Efficient Document Retrieval with Vision Language Models," *arXiv preprint arXiv:2407.01449*, 2024.
[5] Z. Hu, et al., "REVEAL: Retrieval-Augmented Visual-Language Pre-Training with Multi-Source Multimodal Knowledge Memory," in *Proc. CVPR*, 2023.
[6] R. Nogueira, et al., "Document Expansion by Query Prediction," *arXiv preprint arXiv:1904.08375*, 2019.
[7] Y. Gao, et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.
[8] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information processing & management*, vol. 24, no. 5, pp. 513–523, 1988.
[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
[10] A. Vaswani, et al., "Attention is all you need," in *Advances in neural information processing systems*, 2017.

[11] J. Devlin, et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL*, 2019.
[12] L. Gao, et al., "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proc. ACL*, 2023.
[13] J. Chen, et al., "MuRAG: Multimodal Retrieval-Augmented Generator," *arXiv preprint*, 2022.
[14] M. Yasunaga, et al., "Retrieval-Augmented Multimodal Language Modeling," in *Proc. ICML*, 2023.
[15] H. Liu, et al., "Visual Instruction Tuning," in *Proc. NeurIPS*, 2023.